Sistema de Control de Inventario en Python

Carrera: Tecnicatura Universitaria en Programación a Distancia

Asignatura: Programación 1

Universidad Tecnológica Nacional (UTN)

Estudiante: Iván Ramírez

1. INTRODUCCIÓN Y OBJETIVO DEL PROYECTO

El presente desarrollo surge de la necesidad de digitalizar la gestión de productos y stock de una ferretería local. El sistema resuelve el problema del control manual mediante una aplicación de consola modularizada y persistente en memoria de ejecución, permitiendo registrar herramientas, consultar su disponibilidad, emitir reportes de stock crítico y procesar ventas o ingresos de mercadería en tiempo real.

Toda la solución técnica ha sido diseñada bajo estrictas restricciones de arquitectura modular, ausencia total de variables globales y clases, garantizando robustez frente a errores mediante el uso de excepciones (try/except y raise) y estructuración de datos basada exclusivamente en listas de diccionarios.

2. CONCEPTOS FUNDAMENTALES APLICADOS AL CÓDIGO

A continuación se detalla la justificación teórica y práctica de las estructuras y conceptos utilizados en la solución desarrollada:

2.1. Estructura de Datos (Lista de Diccionarios)

Se utiliza una única colección principal denominada inventario, la cual se crea dentro de la función main() y se transfiere por parámetro a lo largo del sistema.

- **Justificación:** Representa entidades compuestas de la vida real (cada herramienta cuenta con dos propiedades clave: 'herramienta' de tipo cadena y 'cantidad' de tipo entero).
- **Vinculación con el código:** En ferreteria/operaciones/carga.py, la incorporación de nuevos productos se realiza directamente en la colección principal mediante la instrucción:
- inventario.append({'herramienta': nombre_valido, 'cantidad': stock_inicial})

Esto evita crear colecciones o diccionarios sueltos fuera del flujo de actualización.

2.2. Modularización por Funciones y Paquetes

El proyecto se divide jerárquicamente en módulos (.py) dentro de un paquete principal denominado ferreteria, el cual contiene subpaquetes (interfaz, operaciones y utilidades).

- **Justificación:** Cumple con el principio de responsabilidad única (*Single Responsibility Principle*), facilitando la mantenibilidad y la depuración aislada de fallas.
- **Vinculación con el código:**
    - ferreteria/interfaz/menu.py: Contiene mostrar_menu(), encargada únicamente del pintado de la interfaz.
    - ferreteria/utilidades/validaciones.py: Centraliza validar_nombre() y buscar_herramienta().
    - ferreteria/operaciones/: Divide la lógica por tipo de proceso (carga.py, consultas.py, stock.py).

2.3. Transferencia de Datos por Parámetro (Referencia y Mutabilidad)

En Python, las listas y los diccionarios son objetos mutables.

- **Justificación:** Al pasar inventario a cualquier función (como cargar_herramientas(inventario)), se transfiere una referencia a la misma dirección de memoria compartida.
- **Vinculación con el código:** Los cambios realizados mediante .append() o modificaciones directas como producto['cantidad'] -= unidades impactan en tiempo real en la variable local inventario declarada en main(), sin requerir declaraciones globales ni retorno explícito de la lista.

2.4. Estructuras de Control (Condicionales y Bucles)

- **Bucles while:** Utilizados para iteraciones indeterminadas, como el menú persistente en main() (while opcion != 7:) y el reintento de entrada de datos válidos dentro de la Opción 1.
- **Bucles for:** Empleados para iteraciones determinadas sobre secuencias, como el recorrido de herramientas registradas en mostrar_inventario() (for producto in inventario:) o el ciclo de carga múltiple (for _ in range(cantidad_cargas):).
- **Condicionales (if / elif / else):** Utilizados para bifurcaciones de reglas de negocio y selección del menú principal.

2.5. Manejo de Errores y Excepciones (try / except y raise)

El sistema diferencia los errores de conversión de los errores de reglas de negocio:

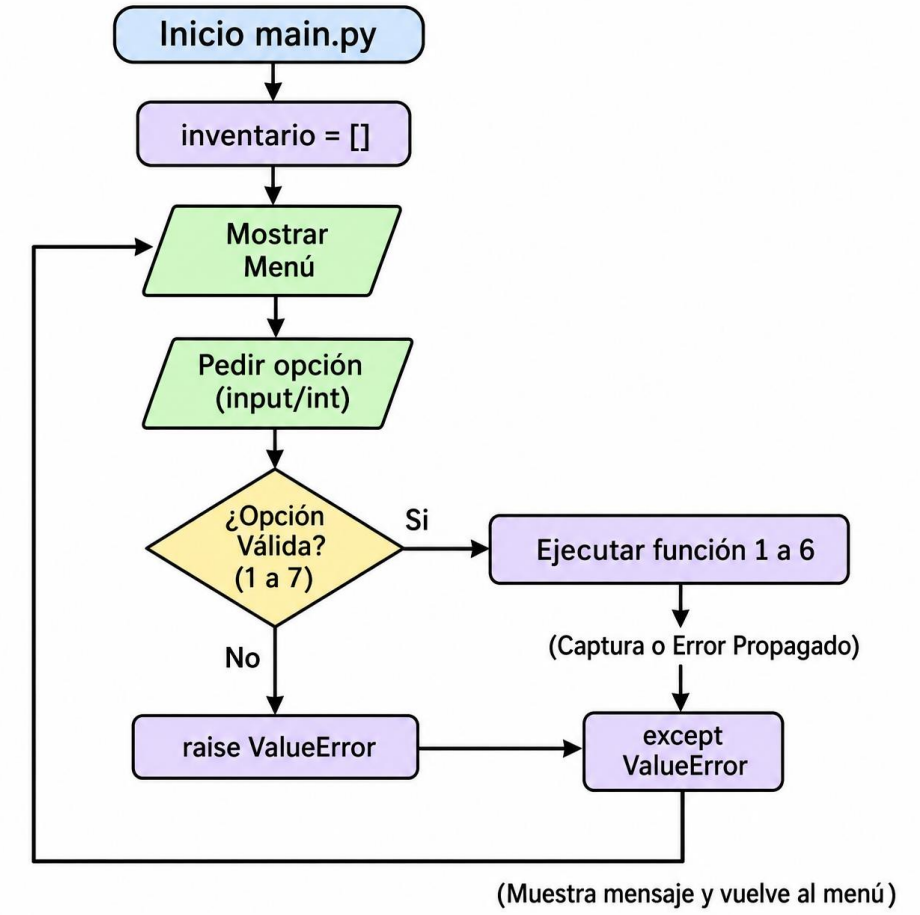
- **Entradas inválidas de formato:** Se capturan automáticamente con bloques try/except ValueError al invocar int().
- **Reglas de negocio incumplidas:** Se fuerzan explícitamente usando raise ValueError("Mensaje descriptivo").
- **Diferencia de propagación entre Opción 1 y Opción 5:**
    - En la **Opción 1** (cargar_herramientas), las excepciones son capturadas dentro de la misma función mediante un bucle interno para permitir al usuario reintentar ingresar el dato sin perder la herramienta que estaba cargando.
    - En la **Opción 5** (alta_producto), las excepciones **no se capturan en la función**, permitiendo que asciendan hasta el bloque try/except de main(), retornando de inmediato al menú principal sin incorporar datos incompletos.

3. LENGUAJE, ENTORNO Y RECURSOS DE DESARROLLO

- **Lenguaje de Programación (Python 3.10+):** Se eligió Python por su expresividad, manejo nativo de tipos de datos complejos (listas y diccionarios) y potencia en el control de excepciones de tipo ValueError.
- **Entorno de Desarrollo Integrado (VS Code - Visual Studio Code v1.85+):** Seleccionado como IDE por su sólida integración con el intérprete de Python, la claridad de su terminal integrada y facilidades de depuración visual.
- **Archivos .py del proyecto y Punto de Entrada:** El proyecto consta de 8 archivos .py. El archivo de entrada que debe ejecutarse para iniciar la aplicación es exclusivamente **main.py**, ubicado en la raíz del proyecto.

4. DIAGRAMA DE FLUJO DEL SISTEMA

4.1. Flujo General del Menú Principal y Excepciones



4.2. Flujo Comparativo de Carga y Manejo de Errores (Opción 1 vs Opción 5)

- **Opción 1 (Reintento Interno):** input nombre —► ¿Inválido/Duplicado? —► **Sí:** except —► Reintenta pedir Nombre.

input stock —► ¿Inválido/Negativo? —► **Sí:** except —► Reintenta pedir Stock (Conserva nombre).

Válido: inventario.append(...)

- **Opción 5 (Propagación a main):** input nombre —► ¿Inválido/Duplicado? —► **Sí:** raise ValueError —► Sale a main() (Volver al menú).

input stock —► ¿Inválido/Negativo? —► **Sí:** raise ValueError —► Sale a main() (Volver al menú).

Válido: inventario.append(...)